应用报告
C2000 器件安全引导

Pramod Prabhakara, Karthik Rajakumar, Christopher Chiarella, Matt Kukucka, and Ronak Harsora

摘要

本应用报告介绍如何使用 F2838x 器件上的 Secure Flash Boot 功能，在从 Flash 启动应用程序时，先对引导代码进行身份验证，从而在实际执行代码前增加一层安全保护。本文档也适用于 F28003x、F280013x、F280015x、F28P55x、F28P551x、
和 F28P65x 器件系列。

目录

1 引言 2
2 安全闪存引导概述 2
3 CMAC 身份验证 3
4 安全闪存引导选项 3
5 安全闪存引导流程 4
6 C2000Ware 示例详解 5
7 对超过 16 KB 的闪存代码进行身份验证 7
8 调试资源 8
9 补充信息与注意事项 8
10 C2000 CMAC 算法与 OpenSSL 的对齐 9
10.1 C28x 存储器和二进制文件的字节顺序 9
10.2 闪存二进制文件的字节顺序 9
10.3 CMAC 密钥字节顺序 10
10.4 CMAC 输出对齐流程 10
10.5 示例演算 11
10.6 差异总结 12
11 参考文献 12
12 修订历史 13

插图清单

图 3-1：CMAC 运算 3
图 6-1：启用 CMAC Hex 工具时的 CPU1 示例属性 6

表格清单

表 2-1：不同器件子系统的安全闪存引导概述 2
表 4-1：安全闪存引导模式配置详情 3
表 8-1：安全闪存引导调试情形 8
表 10-1：CMAC 密钥字节顺序对应关系 10
表 10-2：C2000 与 OpenSSL CMAC 流程对比 12

商标

C2000™ 是 Texas Instruments 的商标。Arm® 是 Arm Limited（或其子公司）在美国及其他地区的注册商标。所有商标均归其各自所有者所有。

Copyright © 2026 Texas Instruments Incorporated

1 引言

TMS320F2838x 是一款高性能 32-bit 浮点实时微控制器，面向工业电机驱动、太阳能逆变器与数字电源、电动汽车与交通运输、传感和信号处理等先进闭环控制应用。该器件采用双核 C28x 架构，并新增连接管理器（CM），可卸载关键通信任务，显著提升系统性能。实时微控制器的高效硬件为强大应用提供了基础，而具体应用的功能则由代码实现。器件内置的双区代码安全模块（DCSM）包含 Secure Boot 功能，可帮助用户构建安全方案，防止未经授权的更新代码运行于应用程序中。

DCSM 实现了双安全区机制：在 Flash 一次性可编程（OTP）区域中写入的安全配置决定可受保护资源（Flash 扇区和 RAM）归属 Zone1、Zone2 还是 Unsecure。除此之外，Zone1/Zone2 资源还可配置为 EXEONLY，此时该区域仅允许代码执行。

更多详情请参见 *DCSM* 章（*TMS320F2838x Technical Reference Manual* [1]）。

2 Secure Flash Boot 概述

与应用程序 Flash 引导相关的一项 DCSM 功能，是在执行前验证 Flash 中的用户应用程序代码。这样可确认代码写入 Flash 存储器后未被篡改，从而确保其完整性。对于 Zone1 的 EXEONLY Flash 扇区，此功能可为关键用户应用程序代码提供额外的安全保护。Secure Flash Boot 在传统 Flash 引导选项之外，还提供一组附加引导选项。

Secure Flash Boot 使用 128-bit AES-CMAC 身份验证算法处理应用程序代码内容，并返回通过/失败状态；只有验证成功后才会继续执行应用程序代码。表 2-1 概述了该功能在器件不同子系统中的实现方式。每个 CPU 子系统的 BootROM 都会启动对该子系统应用程序代码前 16KB 的身份验证，称为主安全引导（Primary Secure Boot）。对每个 CPU 子系统中超出前 16KB 部分的应用程序代码进行身份验证，称为扩展安全引导（Extended Secure Boot）。此步骤可由通过预先身份验证的应用程序代码选择性启动。

由于 Secure Flash Boot 期间要执行 CMAC 身份验证算法，相比普通（非安全）Flash 引导，启动序列需要更长时间才能进入用户应用程序。请注意，器件 CM 核的 Secure Flash Boot 耗时少于 CPU1 或 CPU2 的安全引导实现，因为 CM 使用硬件 AES 加速器。

表 2-1. 器件各模块的 Secure Flash Boot 概述

子系统（内核）	安全引导功能	CMAC 算法实现	对前 16 KB 的闪存引导代码进行身份验证所需时间
CPU1 SS（C28_1）	是	软件（Secure ROM 工具）+ AES ROM 表	~400 ms（INTOSC 运行频率为 10 MHz）
CPU2 SS（C28_2）	是	软件（Secure ROM 工具）+ AES ROM 表	~20 ms（PLL 运行频率为 200 MHz）
CMSS（CM4）	是	软件（Secure ROM 工具）+ 硬件 AES 加速器	~6 ms（PLL 运行频率为 125 MHz）

Copyright © 2026 Texas Instruments Incorporated

3 CMAC 身份验证

基于 AES 的基于密码的消息认证码（CMAC）算法会根据一段输入数据生成身份验证标签。输入数据每次以 128 bits 为一组，与 128-bit CMAC 密钥一起送入加密引擎或软件（取决于 CPU 子系统）。该密钥存储在 CPU1 USER OTP 的 0x78018-0x7801F 地址范围内，供器件所有内核上的 CMAC 身份验证算法使用。加密引擎或软件会对整个输入数据块运行 CMAC 算法，并生成最终的 128-bit 身份验证标签。

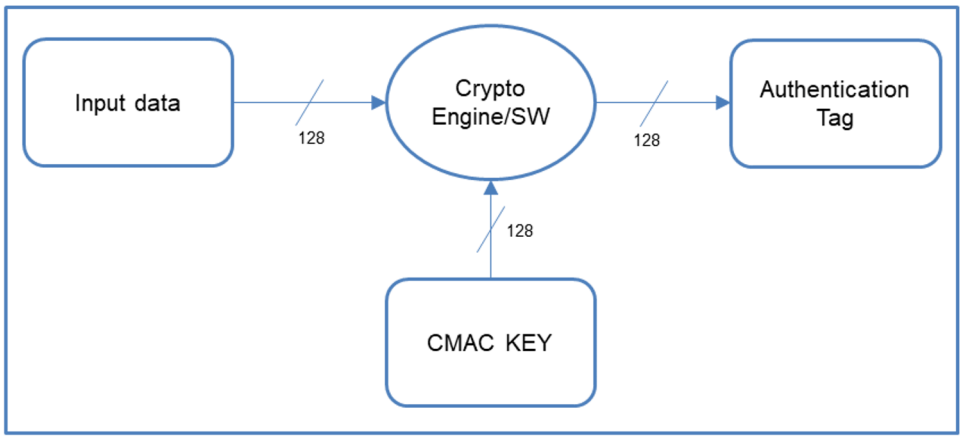


图 3-1. CMAC 运算

4 Secure Flash Boot 选项

表 4-1 列出了三个内核可用的不同主 Secure Flash Boot 选项及其相应配置。由应用程序代码启动的扩展安全引导见第 7 节。更多详情请参见 *ROM Code and Peripheral Booting* 章（*TMS320F2838x Technical Reference Manual* [1]）。

表 4-1. Secure Flash Boot 模式配置详情

安全引导选项 (1)	**BOOTDEFx/ BOOTMODE** 值 (2)	闪存入口点 (3)	CPU1/CPU2 入口地址	CPU1/CPU2 128 位基准 CMAC 标签存储位置	CM 入口地址	CM 128 位基准 CMAC 标签存储位置
选项 0	0x0A	扇区 0	0x00080000	0x00080002	0x00200000	0x00200004
选项 1	0x2A	扇区 4	0x00088000	0x00088002	0x00210000	0x00210004
选项 2	0x4A	扇区 8	0x000A8000	0x000A8002	0x00250000	0x00250004
选项 3	0x6A	扇区 13	0x000BE000	0x000BE002	0x0027C000	0x0027C004

(1) CPU1/CPU2/CM 可分别选择 Secure Boot 选项。(2) 对于 CPU1，BOOTDEFx 字段位于 Zx-BOOTDEF-LOW/Zx-BOOTDEF-HIGH CPU1 USER OTP 存储器位置。对于 CPU2/CM，BOOTMODE 字段分别位于 CPU1TOCPU2IPCBOOTMODE/CPU1TOCMIPCBOOTMODE 寄存器中，并由 CPU1 应用程序代码写入。(3) Secure Boot 功能仅适用于 Zone1，因此所选 Flash 扇区必须配置为 Zone1-EXEONLY。此外，无论扇区大小如何，主 Secure Flash Boot 都只处理所选扇区的前 16KB。例如，

Sector 4 和 Sector 8 的大小均为 64KB，但只有前 16KB 会纳入处理。

Copyright © 2026 Texas Instruments Incorporated

5 Secure Flash Boot 流程

器件上的 Secure Flash Boot 分为两个步骤：

1. 生成身份验证标签 — 在设备外部生成映像时完成。

   a. C2000™ 或 Arm® hex 实用程序使用输入 CMACKEY 和 CMAC 应用数据结构，对 Flash 引导代码映像运行 CMAC 算法；这些数据结构会为 golden CMAC 身份验证标签保留存储空间。有关 hex 实用程序的详细信息，请参见 [3] 和 [4]。b. 生成的 golden CMAC 标签会嵌入 hex 文件中，位置见表 4-1。c. 将包含 golden CMAC 标签的 hex 映像编程到相应 Flash 扇区。d. 按照表 4-1 选择适当的 Secure Flash Boot 模式，并将其编程到 CPU1 USER OTP。

2. 在 Flash 中验证应用程序引导代码 — 在器件执行 Secure Flash Boot 时完成。

   a. 配置 BOOTDEFx/BOOTPINCONFIG 字段，以按照表 4-1 选择 Secure Flash Boot 选项。器件复位后启动，并对指定 Flash 扇区运行 CMAC 算法。b. 将 CMAC 算法生成的标签与预先编程位置中的 golden CMAC 标签进行比较。c. 标签匹配后，引导流程跳转到已验证的 Flash 代码并开始执行。d. 标签不匹配时，CPU1/CPU2/CM 会采取不同操作：i. 对 CPU1，器件会复位（代码持续循环，Watchdog 超时后自动发出 XRSn）。ii. 对 CPU2，CPU2TOCPU1IPCBOOTSTS 寄存器中的 Secure Boot 失败标志会置位，

   随后向 CPU1 发送包含 Secure Flash CMAC 错误代码的 IPC 命令，CPU2 引导代码进入循环，等待 CPU1 采取必要措施。CPU2 还会在地址 0x0000 0002 处保留 CPU2TOCPU1IPCBOOTSTS 寄存器副本。iii. 对 CM，CMTOCPU1IPCBOOTSTS 寄存器中的 Secure Boot 失败标志会置位，随后向 CPU1 发送包含 Secure Flash CMAC 错误代码的 IPC 命令；CM 引导代码进入循环，等待 CPU1 采取必要措施。CM 还会在地址 0x2000 0000 处保留 CMTOCPU1IPCBOOTSTS 寄存器副本。

Note

The CMAC algorithm, while calculating the authentication tag on the image and also while authenticating the image, treats the memory addresses containing the golden tag as all ones.

Copyright © 2026 Texas Instruments Incorporated

6 C2000Ware 示例详解

C2000Ware [2] 中提供了一个示例，演示 Secure Flash Boot 应用的配置。该示例包含适用于每个内核的 Secure Flash Boot 应用工程，还详细说明如何对超出 Secure Flash Boot 入口地址 + 16KB 范围的 Flash 代码进行身份验证。第 7 节将进一步介绍此自定义 Flash 范围身份验证功能。此示例假定待验证的 Flash 扇区已预先配置为 Zone 1 EXEONLY，并使用默认 CMACKEY 进行身份验证。有关在 CPU1 USER OTP 中编程自定义 CMACKEY 和其他 DCSM 设置的详细信息，请参见 [1] 和 [2]。

C2000Ware 位置：<C2000Ware_Install_Directory>/driverlib/f2838x/examples/c28x/boot

工程名称：

- boot_ex1_cpu1_cpu2_cm_secure_flash_cpu1
- boot_ex1_cpu1_cpu2_cm_secure_flash_cpu2
- boot_ex1_cpu1_cpu2_cm_secure_flash_cm

包含的文件：

- 源文件——包含主应用程序代码。例如：boot_ex1_cpu1_cpu2_cm_secure_flash_cpu1.c
- HEX 链接器命令文件——向 C2000 或 Arm hex 实用程序提供 Flash 存储器的总长度等信息。例如：boot_ex1_flash_hex_lnk_cpu1.cmd
- CMAC 密钥文本文件——向 C2000 或 Arm hex 实用程序提供用户 CMAC 密钥。示例：boot_ex1_user_cmac_key.txt。有关 cmac_key 格式的详细信息，请参见 [3] 和 [4]。

示例运行方法：

1. 将应用程序加载到 CPU1 Flash（CPU2 和 CM 应用程序也要加载）。a. 加载 *.hex 文件，不要加载 *.out 文件。

2. 断开连接，然后仅重新连接到 CPU1。3. 将器件配置为在启动时执行 Secure Flash Boot：a. 仿真引导（示例/开发阶段推荐）i. 在 CCS 内存窗口中，将 BOOTPINCONFIG 地址（0x0D00）设为 0x5AFFFFFF，并将 BOOTDEF 地址（0x0D04）设为 0x0000000A。b. 独立引导（部署时推荐）i. 将对应于 BOOTPINCONFIG 和 BOOTDEF 的 CPU1 USER OTP 位置编程。详情请参见 [1]。

4. 通过 CCS 复位 CPU1，然后单击 Resume。
5. 观察 controlCARD 上的 LED 指示灯。
   a. 当三个内核均成功完成 Secure Boot 并验证其完整 Flash 存储器内容时，三个 LED（每个内核对应一个）将闪烁。

6. 如果未使用 controlCARD，请根据 CPU 子系统留意以下 GPIO 的翻转：

   a. CPU1 – GPIO31 b. CPU2 – GPIO34 c. CM – GPIO145

Copyright © 2026 Texas Instruments Incorporated

生成 Golden CMAC 标签时对应用程序代码的要求：

Secure Flash Boot 选项 0 的 CPU1/CPU2 Golden CMAC 标签存储器分配

```
// Implementation for CPU1/CPU2 #pragma RETAIN(c
mac_sb_1) #pragma LOCATION (cmac_sb_1, 0x08000
2) const char cmac_sb_1[8] = {0};
```

Secure Flash Boot 选项 0 的 CM Golden CMAC 标签存储器分配

```
#pragma RETAIN(cmac_sb_1) #pragma LOCATION (cma
c_sb_1, 0x00200004) const uint8_t cmac_sb_1[16] = {0};
```

常量 char/unsigned integer 定义用于分配 Golden CMAC 标签的存储器。更多信息请参见第 6 节中的示例：

- 变量名称必须为以下名称之一：cmac_sb_1、cmac_sb_2、cmac_sb_3、cmac_sb_4。有关 C28x 的更多详情，请参见 *TMS320C28x Assembly Language Tools User's Guide* [3]；有关 CM 的更多详情，请参见 *ARM Assembly Language Tools User's Guide* [4]。
- 使用 LOCATION pragma 指定 Golden CMAC 标签在目标身份验证范围内的地址。对于 CPU1/CPU2，该地址必须为入口地址 + 2；对于 CM，该地址必须为入口地址 + 4。
- 将该变量初始化为零。

使用 HEX 实用程序的应用程序设置：

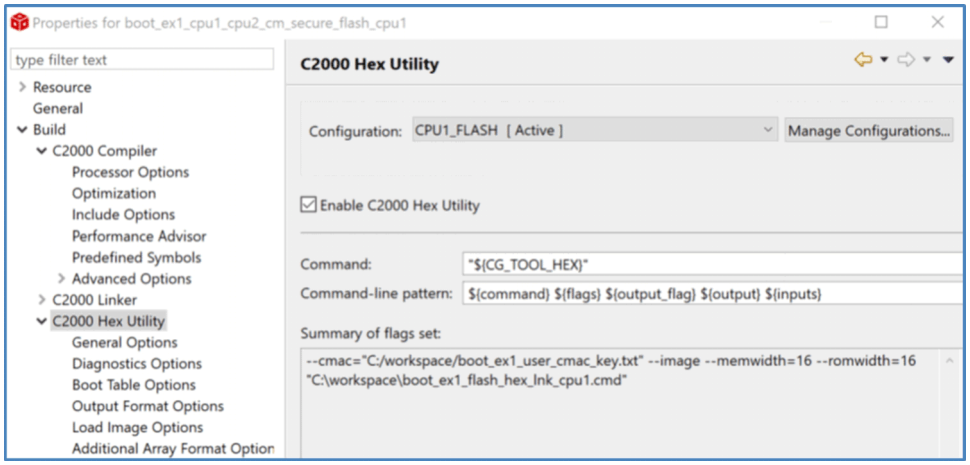


图 6-1。 启用 HEX 实用程序进行 CMAC 运算时 CPU1 示例的属性

每个内核的工程都会启用 hex 实用程序来生成 Golden CMAC 标签（见图 6-1）。包含的标志如下：

- “--cmac”用于指定用户 CMAC 密钥文本文件的路径。
- “--image”、“--memwidth”和“--romwidth”，其中 mem/rom 宽度需要设置为相应值。CPU1/CPU2 应设为 16，CM 应设为 8。
- 对应内核的 Flash HEX 链接器命令文件路径。

Copyright © 2026 Texas Instruments Incorporated

7 验证超出 16 KB 范围的 Flash 代码

Secure Flash Boot 仅对从入口地址开始的 Flash 扇区前 16 KB 进行身份验证。若要验证其他 Flash 扇区，用户应用程序必须直接调用 Secure Flash Boot CMAC API。

借助 hex 实用程序，可为四个 Flash 入口地址 + 16KB 各自生成 Golden CMAC 标签，并为 1 个自定义 Flash 范围生成标签。自定义 Flash 范围可配置，因此能够对任意指定的地址范围执行 CMAC 身份验证；如有需要，该范围可以覆盖所有 Flash 扇区。

用于自定义 Flash 范围身份验证的 CPU1/CPU2 应用程序 CMAC 结构

```
struct CMAC_TAG {   cha
r tag[8];   uint32_t start;
uint32_t end; }
```

用于完整 Flash 范围身份验证的 CPU1/CPU2 Golden CMAC 标签存储器分配

```
#pragma RETAIN(cmac_all) #pragma LOCATION (cmac_all, 0x087002
) const struct OMAC_TAG cmac_all = {{0}, 0x0, 0x0};
```

用于自定义 Flash 范围身份验证的 CM 应用程序 CMAC 结构

```
struct CMAC_TAG {   uint8
_t tag[16];   uint32_t start;
uint32_t end; }
```

用于完整 Flash 范围身份验证的 CM Golden CMAC 标签存储器分配

```
#pragma RETAIN(cmac_all) #pragma LOCATION (cmac_all, 0x002040
04) const struct OMAC_TAG cmac_all = {{0}, 0x0, 0x0};
```

创建名为“cmac_all”的结构体，以定义自定义 CMAC 身份验证范围。

- 使用 LOCATION pragma 指定 Golden CMAC 标签在预期身份验证范围内的任意地址。（Golden CMAC 标签越接近身份验证起始地址，CMAC 算法执行得越快。）
- “tag”结构体成员必须始终初始化为零。
- 初始化“start”和“end”结构体成员以设置自定义范围。如果两者均为零，则会对器件内核 Flash 存储器的全部范围进行身份验证，其中包括 Primary Secure Boot 的 16KB 存储器范围。
- 有关应用程序 CMAC 变量/结构体的更多详情，请参见 [3] 和 [4]。

CPU1 Secure Flash CMAC 身份验证 API

```
applicationCMACStatus = CPU1BROM_calculateCMAC (CMAC_AUTH_START_ADDRESS,
CMAC_AUTH_END_ADDRESS,                          CMAC_AUTH_TAG_ADDRESS);
```

在应用程序中，将 F2838x 安全区代码符号库添加到工程（位于 C2000Ware [2] 的 <C2000Ware_Install_Directory>/libraries/boot_rom/f2838x），并针对相应内核调用 Secure Flash Boot CMAC API。上文给出了 CPU1 的 API 调用示例。

- CMAC_AUTH_TAG_ADDRESS 必须与 LOCATION pragma 中提供的地址一致。
- CMAC_AUTH_START_ADDRESS 和 CMAC_AUTH_END_ADDRESS 必须与“cmac_all”结构体中提供的地址一致。此外，起始地址和结束地址都必须按 128 bits 对齐。

Copyright © 2026 Texas Instruments Incorporated

例如，验证 F2838x CPU1 Flash 的完整范围时，应将下列地址设为起始地址和结束地址：
- 起始地址：0x00080000
- 结束地址：0x000C0000

随后，在用户应用程序中检查 Secure Flash Boot CMAC API 返回状态并作相应处理。
有关 Secure Flash CMAC 身份验证 API 的详细信息，请参见 [1] 中的 ROM Code and Peripheral Booting 章节。

注意：实际结束地址为 0x000BFFFF；为满足 128 位对齐要求，参数必须设为 0x000C0000。

8 调试资源

表 8-1. Secure Flash Boot 调试场景

场景	行为
CPU1 安全闪存引导失败	独立引导：器件复位。 仿真引导：CPU1 在地址范围 0x3FB13C – 0x3FB142 内暂停。
CPU2 安全闪存引导失败	CPU2TOCPU1IPCBOOTSTS [1] 寄存器的 Bit21 位置将置位。 CPU2 向 CPU1 发送安全闪存 CMAC 错误 IPC 命令。
CM 安全闪存引导失败	CMTOCPU1IPCBOOTSTS [2] 寄存器的 Bit21 位置将置位。CM 向 CPU1 发送安全闪存 CMAC 错误 IPC 命令。
CPU1 安全引导是否成功?	CPU1 BootROM 状态的 Bits7:0 位于地址 0x0000 0002 处的值为 0x3。
CPU2 安全引导是否成功?	CPU2TOCPU1IPCBOOTSTS 的 Bits7:0 [1] 寄存器的值为 0x3。
CM 安全引导是否成功?	CMTOCPU1IPCBOOTSTS 的 Bits7:0 [2] 寄存器的值为 0x3。

1. 相同信息也保存在 CPU2 地址 0x0000 0002 处。2. 相同信息也保存在 CM 地址 0x200 0 0000 处。

9 其他信息和注意事项

- 虽然不建议这样做，但如果在 CPU2/CM 上执行 Secure Flash Boot、而未在 CPU1 上执行，则在解除 CPU2/CM 复位之前，必须由 CPU1 对 Z1 OTP CMACKEY 执行一次空加载。具体做法是读取 CPU1 USER OTP 的 0x7801 8-0x7801F 地址。
- 同样，如果在 CPU1 上未运行 Secure Flash Boot 模式而使用 CMAC 身份验证 API，则应 先执行一次空加载：由 CPU1 读取 Z1 OTP CMACKEY，然后再调用这些 API。
- 在 CPU1 上使用 Secure Flash Boot 时，建议不要在 BOOTDEF 表中将同一扇区配置为普通（非安全）Flash Boot 模式。
- 验证超出 16 KB 范围的 Flash 代码时：
  – 128-bit Golden CMAC 标签必须存储在参与计算的存储器地址范围内。– Golden CMAC 标签的起始地址必须按 32-bit 边界对齐。
- 由于引导模式设置存储在一次性可编程存储器（OTP）中，建议先使用仿真引导模式进行试验，再最终确定引导相关配置。有关仿真引导的更多信息，请参见 [1] 中的 *ROM Code and Peripheral Booting TMS320F2838x Technical Reference Manual* 章。

Copyright © 2026 Texas Instruments Incorporated

10 将 C2000 CMAC 算法与 OpenSSL 对齐

C2000™ hex 实用程序（hex2000）会对消息数据（CMAC 计算前）以及生成的身份验证标签（嵌入 Flash 映像前）执行专有的字节重排变换，本文将其称为“swapwords”。第 10.2 节对此作了详细说明。

OpenSSL 的 AES-128-CMAC 实现直接处理未经修改且连续的字节流，不执行此类重排，因此直接对原始二进制文件调用 *openssl dgst -mac* cmac 命令，所得结果将与 C2000 生成的 Golden CMAC 标签不匹配。本节将说明所需的精确操作步骤，使 OpenSSL 的输出与 C2000 保持一致，从而让开发者能够使用标准开源加密工具独立验证或复现 Golden CMAC 标签。

此对齐流程仅供开发和验证使用，不能替代生产映像生成时使用的 hex2000 实用程序。有关 *hex2000 --cmac* 标志和密钥文件格式的详细信息，请参见 [3] 和 [4]。

10.1 C28x 存储器与二进制文件的 字节顺序

C28x CPU 按 16-bit 字寻址（不能按字节寻址）。每个 C28x 字宽为 16 bits。在二进制文件中，每个 16-bit 字均按小端序存储（低字节位于较低的文件偏移处）。

32-bit 值在二进制文件中的表示方式

存储在字地址 N 处的 C28x uint32_t 值 0xAABBCCDD：

```
C28x word address N:   value = 0xCCDD  (low 16 bits)
C28x word address N+1: value = 0xAABB  (high 16 bits)

Binary file bytes:
  offset 2k+0: 0xDD  ← low  byte of word[N]
  offset 2k+1: 0xCC  ← high byte of word[N]
  offset 2k+2: 0xBB  ← low  byte of word[N+1]
  offset 2k+3: 0xAA  ← high byte of word[N+1]
```

因此，uint32_t 0xAABBCCDD 在二进制文件中表示为：DD CC BB AA

示例：codestart 分支指令

位于 C28x 地址 0x080000 处的“LB 0x081BC8”指令：

```
C28x 字地址 0x080000：0x4800 → 二进制字节：00 48；C28x 字地址
0x080001：0x1BC8 → 二进制字节：C8 1B

二进制文件偏移 0 处：00 48 C8 1B
```

10.2 Flash 二进制数据的字节顺序

hex2000 实用程序不会将 Flash 二进制数据直接输入 AES-128-CMAC 算法。计算身份验证标签之前，它会应用 *swapwords* 变换，对二进制数据中每个 32-bit 分组内的两个 16-bit 字进行交换。

对于二进制文件中每组连续的 4 个字节 [b0, b1, b2, b3]，有：

- [b0, b1] = 对应地址 N 处 C28x 字的 [低字节，高字节]
- [b2, b3] = 对应地址 N+1 处 C28x 字的 [低字节，高字节]

```
swapwords([b0, b1, b2, b3]) = [b2, b3, b0, b1]
```

此操作会交换每个 32-bit 分组内的两个 16-bit 字，同时保持每个字节仍按自然的小端序排列。等效做法是将每对 C28x 字按相反顺序读取（先读取 N+1 处的高位字，再读取 N 处的低位字）。此操作具有自反性：对任意输入连续应用两次该变换，都会还原为原始数据。

Copyright © 2026 Texas Instruments Incorporated

在变换后的数据上计算出 AES-128-CMAC 标签后，hex2000 会再次对这 16-byte 标签应用相同的 swapwords 变换，然后将其嵌入 Flash 映像中 Golden CMAC 标签所在的位置；该位置为距 Flash bank 入口地址 +0x4 的字节偏移，对应 C28x 字地址 +0x2。使用 OpenSSL 时必须复现这两次变换，才能生成与 hex2000 生成结果逐字节完全一致的已存储标签。

10.3 CMAC 密钥的字节顺序

128-bit CMAC 密钥以 4 个 32-bit 寄存器的形式配置在 DCSM Zone 1 OTP 中（DCSM_Z1_CMACKEY0 至 DCSM_Z1_CMACKEY3）；CMACKEY0 保存最高有效 32 bits，CMACKEY3 保存最低有效 32 bits。

hex2000 密钥文本文件会按从最高有效位到最低有效位的顺序连接这 4 个值，并在前面添加“0x”前缀。OpenSSL 的 *-macopt hexkey:<32-hex-key>* 选项可直接接受相同的 32 字符十六进制字符串，无需额外调整字节顺序。表 10-1 展示了 OTP 寄存器值、hex2000 密钥文件格式和 OpenSSL hexkey 参数之间的对应关系。

表 10-1. CMAC 密钥字节顺序对应关系

OTP 寄存器	示例值	用途
DCSM_Z1_CMACKEY0	0x2b7e1516	密钥位 [127:96]（MSW）
DCSM_Z1_CMACKEY1	0x28aed2a6	密钥位 [95:64]
DCSM_Z1_CMACKEY2	0xabf71588	密钥位 [63:32]
DCSM_Z1_CMACKEY3	0x09cf4f3c	密钥位 [31:0]（LSW）
hex2000 密钥文件	0x2b7e151628aed2a6abf7158809cf4f3c	单行文本，带 0x 前缀
OpenSSL hexkey 参数	2b7e151628aed2a6abf7158809cf4f3c	数值相同，不带 0x 前缀

10.4 CMAC 输出对齐流程

使用 OpenSSL 复现 hex2000 的 Golden CMAC 标签需要经过 5 个步骤，下面逐一说明。

步骤 1 — 获取签名前的二进制文件。

首先准备尚未进行 CMAC 签名的 16 KB（0x4000-byte）Flash 二进制文件。此时，字节偏移 0x4 处的 16-byte Golden CMAC 标签占位符（cmac_sb_1）必须初始化为全零，这与第 6 节所述的应用程序源代码要求一致。其余 16 KB 范围应包含预期写入 Flash 的内容，未编程区域填充 0xFF。

步骤 2 — 屏蔽 Golden CMAC 标签区域。

CMAC Golden 标签存储在 CMAC 覆盖的 16 KB 区域内。为避免循环依赖（标签依赖其自身），计算 CMAC 时会用全 1（0xFFFF）屏蔽该标签区域。C28x 器件读取已擦除的 Flash 时，结果为全 1。屏蔽该区域后，CMAC 计算会将标签位置视为已擦除的 Flash，这与编程前 Flash 的初始状态一致。

因此，计算 CMAC 前，应将字节 0x4 至 0x13（含）替换为 0xFF。无论使用 hex2000 还是 OpenSSL 计算，都必须执行此屏蔽步骤；该步骤还
与第 5 节所述行为一致。

步骤 3 — 应用 swapwords 变换，生成 OpenSSL 输入。

对整个经过屏蔽的 16 KB 二进制文件应用 swapwords 字节重排。对于每组连续的 4 个字节 [b0, b1, b2, b3]，将其替换为 [b2, b3, b0, b1]。将生成的 16 KB 字节序列写入临时文件（例如 input_swapped.bin）。该变换后的文件即为 OpenSSL CMAC 命令的直接输入。

步骤 4 — 使用 OpenSSL 计算 AES-128-CMAC。

按以下方式调用 OpenSSL，并替换为相应的 32 字符十六进制密钥字符串：

Copyright © 2026 Texas Instruments Incorporated

使用 OpenSSL 对变换后的输入计算 AES-128-CMAC 的命令

```
openssl dgst -mac cmac \
             -macopt cipher:AES-128-CBC \
             -macopt hexkey:<32-hex-key> \
             -binary input_swapped.bin > aes_tag.bin
```

输出文件 aes_tag.bin 将包含恰好 16 bytes，表示原始 AES-128-CMAC 标签。该值对应 hex2000 在执行最后一次变换之前计算出的内部标签。

步骤 5 — 对标签应用 swapwords，得到存储格式。

对步骤 4 得到的 16-byte AES 标签应用同一 swapwords 变换。生成的 16 个字节即为 hex2000 使用且 Boot ROM 预期的存储格式 Golden CMAC 标签。该值应与签名 Flash 映像中字节偏移 0x4 处由 hex2000 写入的内容一致，也是 Boot ROM 在 C28x 字地址 0x00080002 处的 Golden CMAC 标签位置读取的值，用于

身份验证。

由于 swapwords 具有自反性，Boot ROM 只需再次读取已存储标签并应用一次 swapwords，即可还原原始 AES 标签；随后将其与针对经过屏蔽和变换的 Flash 内容重新计算出的标签进行比较。

10.5 示例演算

以下示例使用 NIST AES-128 测试密钥 2b7e151628aed2a6abf7158809cf4f3c，并将其用于一个最小 Flash 映像。应用程序的第一个字（偏移 0x0）是“LB 0x081BC8”分支指令，由两个 C28x 字“0x4800”和“0x1BC8”汇编而成。偏移 0x4 处的 Golden CMAC 标签占位符初始化为零。偏移 0x14 及之后的字段处于擦除状态（0xFF）。其余 16 KB 范围均填充 0xFF。

签名前的二进制文件（显示前 24 bytes，其余部分为 0xFF）：

```
Offset 0x0000:  00 48 C8 1B   (codestart: LB 0x081BC8)
Offset 0x0004:  00 00 00 00   (cmac_sb_1, initialized to zero)
Offset 0x0008:  00 00 00 00   (cmac_sb_1, continued)
Offset 0x000C:  00 00 00 00   (cmac_sb_1, continued)
Offset 0x0010:  00 00 00 00   (cmac_sb_1, continued)
Offset 0x0014:  FF FF FF FF   (erased contents)
```

步骤 2 后 — 使用 0xFF 屏蔽签名区域:

```
Offset 0x0000:  00 48 C8 1B   (unchanged)
Offset 0x0004:  FF FF FF FF   (masked)
Offset 0x0008:  FF FF FF FF   (masked)
Offset 0x000C:  FF FF FF FF   (masked)
Offset 0x0010:  FF FF FF FF   (masked)
Offset 0x0014:  FF FF FF FF   (unchanged)
```

步骤 3 后 — 应用 swapwords（显示前 8 bytes）：

```
Offset 0x0000:  C8 1B 00 48   ([b2,b3,b0,b1] of original offset 0x0000)
Offset 0x0004:  FF FF FF FF   (unchanged; all bytes identical)
```

步骤 4 — OpenSSL AES-128-CMAC 输出（aes_tag.bin，16 bytes）：

```
f2 70 99 41  e1 cf 3c cf  f1 36 7a a3  c4 3e f6 f7
```

步骤 5 后 — 对标签应用 swapwords（存储格式）：

```
99 41 f2 70  3c cf e1 cf  7a a3 f1 36  f6 f7 c4 3e
```


Copyright © 2026 Texas Instruments Incorporated

最终签名后的二进制文件（前 32 bytes）：

```
Offset 0x0000:  00 48 C8 1B
Offset 0x0004:  99 41 F2 70   (stored tag bytes 0-3)
Offset 0x0008:  3C CF E1 CF   (stored tag bytes 4-7)
Offset 0x000C:  7A A3 F1 36   (stored tag bytes 8-11)
Offset 0x0010:  F6 F7 C4 3E   (stored tag bytes 12-15)
Offset 0x0014:  FF FF FF FF
```

上方存储的标签与使用相同密钥对同一二进制文件运行 *hex2000 --cmac* 所得输出一致。已验证该结果与 hex2000 的输出逐字节完全一致。

10.6 差异总结

hex2000 --cmac 签名流程与直接调用 OpenSSL AES-128-CMAC 命令之间，唯一的行为差异是在 AES 运算前后各执行一次 swapwords 变换。两种流程使用的底层加密算法、密钥调度和初始化向量完全相同。表 10-2 汇总了两种流程的完整对应关系，供参考。

表 10-2. C2000 与 OpenSSL CMAC 流程对比

比较项	C2000 实现	OpenSSL 实现
消息预处理	AES 前执行 swapwords(masked_binary)	原始字节流；不做变换
签名区域屏蔽	将字节偏移 0x4 – 0x13 替换为 0xFF	调用 OpenSSL 前必须手动屏蔽
密钥字节序	最高有效位在前（CMACKEY0 || ... || CMACKEY3）	最高有效位在前；顺序相同，无需转换
AES-128-CMAC 算法	NIST SP 800-38B (OMAC1)	NIST SP 800-38B（OMAC1），两者相同
初始化向量	全 0（CMAC 标准中隐含）	全 0（CMAC 标准中隐含）
标签后处理	嵌入前执行 swapwords(aes_tag)	原始 AES 标签输出；需手动执行 swapwords
输出	含已存储标签的签名二进制文件	16 字节原始 AES 标签；仍需进一步处理

注意

swapwords 变换按 32-bit 分组执行，因此经过身份验证的区域长度必须是 4 bytes 的整数倍。16 KB（0x4000-byte）的 C28x Primary Secure Boot 区域符合此要求。按照第 7 节所述，对自定义 Flash 范围执行扩展身份验证时，起始地址和结束地址也必须按 128 bits 对齐，以确保 CMAC 算法无需填充即可正确运行。

11 参考文献

1. Texas Instruments：*TMS320F2838x Real-Time Microcontrollers Technical Reference Manual* 2. 面向 C2000 实时 MCU 的 C2000Ware 3. Texas Instruments：*TMS320C28x Assembly Language Tools v20.2.0.LTS User's Guide* 4. Texas Instruments：*Arm Assembly Language Tools v20.2.0.LTS User's Guide* 5. TMS320F28338D 产品页面

Copyright © 2026 Texas Instruments Incorporated

12 修订历史

从修订版 *（2020 年 9 月）到修订版 A（2026 年 6 月）的变更 页码

- 更新对较新 C2000 器件的 Secure Boot 支持.....1
- 更新全文表格、插图和交叉引用的编号格式.....2
- 新增介绍 C2000 实现与 OpenSSL 对齐方式的章节.....9

Copyright © 2026 Texas Instruments Incorporated

重要声明和免责声明

TI 按“现状”提供技术和可靠性数据（包括数据表）、设计资源（包括参考设计）、应用或其他设计建议、Web 工具、安全信息及其他资源，并且不承担任何缺陷责任。TI 不作任何明示或暗示的保证，包括但不限于适销性、特定用途适用性或不侵犯第三方知识产权的暗示保证。

这些资源面向使用 TI 产品进行设计的专业开发人员。您需独自负责：(1) 为您的应用选择合适的 TI 产品；(2) 设计、验证并测试您的应用；以及 (3) 确保您的应用符合适用标准及其他安全、安保、监管或其他要求。

这些资源如有变更，恕不另行通知。TI 仅允许您将这些资源用于开发使用本文所述 TI 产品的应用。禁止以其他方式复制或展示这些资源。TI 不授予您使用任何其他 TI 知识产权或第三方知识产权的许可。对于因您使用这些资源而产生的任何索赔、损害、成本、损失和责任，TI 不承担责任，您应全额赔偿 TI 及其代表，并使其免受相关损失。

TI 产品受 TI 销售条款、TI 通用质量指南或其他适用条款约束；相关条款可在 ti.com 上查阅，或随 TI 产品一同提供。TI 提供这些资源不会扩大或以其他方式改变 TI 产品适用的保证或免责声明。除非 TI 明确将产品指定为定制产品或客户指定产品，否则 TI 产品均为标准目录通用器件。

对于您可能提出的任何附加或不同条款，TI 均予以反对并拒绝。

重要声明

Copyright © 2026, Texas Instruments Incorporated

Last updated 10/2025